

Generational Programming

When COBOL & VB.Net Matters



VB.NET Evolutions

2002–08

REBUILD

2002 VB.NET launches on the CLR, replacing VB6 with full object orientation and type safety.

2005 Generics, partial classes, and the My namespace accelerate application development.

2008 LINQ, lambdas, XML literals, and type inference bring data into the language.

2010–15

MODERNIZE

2010 Auto-properties, collection initializers, and generic variance reduce boilerplate.

2012 Async/Await and iterators simplify responsive, asynchronous applications.

2015 VB 14 enters the Roslyn compiler era with modernized tooling.

2017–TODAY

STABILIZE

2017–22 VB 15–17 continue alongside Visual Studio and modern .NET.

2023+ Microsoft stable: **Maintained** for core .NET, Windows Forms, libraries, and 'interop—not language enhancements.

TODAY: Maintained & Interoperable. **Focus:** Stability *not* language enhancement.

Sources: Microsoft Learn, “What’s new for Visual Basic” and “Visual Basic language strategy.”

Mono: Beyond Windows

WHY IT MATTERED

.NET PORTABILITY

Mono supplied an open-source CLR, libraries, and tooling that could run many Windows-built .NET assemblies on Linux, macOS, and Unix.

Compatibility was practical—not perfect: unsupported APIs and operating-system differences still required testing.

2001 Mono launches as an independent, open-source .NET implementation.

2007 Mono 1.2.3 completes VB.NET support with a self-hosting compiler, runtime, and class libraries.

2016 **Microsoft becomes steward of Mono through its acquisition of Xamarin.**

2019–24 Original Mono shifts to maintenance: last major release in 2019; **final Microsoft patch in 2024.**

TODAY **For new VB development,** use the **.NET SDK**; migrate active Mono/Xamarin workloads to modern .NET.

Sources: Mono Project — “Visual Basic support,” “FAQ: Technical,” and project home; Microsoft .NET Xamarin support policy.



Installing .NET SDK

Official package managers:

Linux (Ubuntu/Debian/Fedora/CentOS):
Available via standard package managers like apt or dnf (e.g., sudo apt-get install dotnet-sdk-8.0).

macOS:

Available via a native .pkg installer, Homebrew (brew install --cask dotnet-sdk), or MacPorts.

Universal Script: You can use the official dotnet-install.sh script for headless or non-admin installations.



Core CLI Commands

‘dotnet’ everywhere:

01 `dotnet new console` — Creates a new console application.

02 `dotnet build` — Compiles the application.

03 `dotnet run` — Runs the application.



NET SDK & POSIX

Key POSIX Differences

01

File Paths:

Use **Path.Combine** to manage

02

Case Sensitive File System

Caveat file / folder names

03

Line Endings

“\r\n” –v- “\n”